EF 1003
Data Literacy Foundation

07. Data structure
- List, Tuple, Set, Dictionary -

Ukcheol Shin

Robust Physical AI & Robotics Lab., KENTECH

Lecture’s agenda

[List, dictionary, set, tuple]

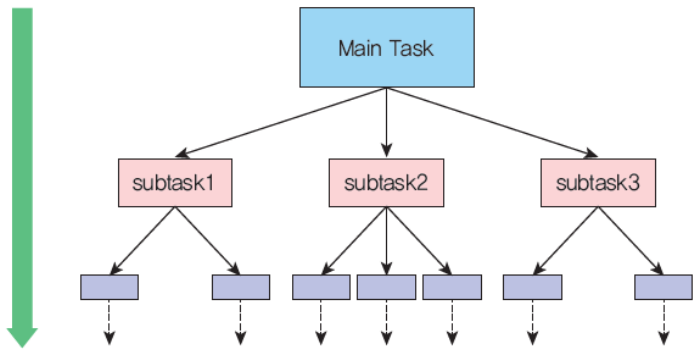
- What is collection?
- Definition
- Creation
- Accessing & updating
- Removing
- Use cases

Programming

- **Algorithm**
  - A set of rules or steps used to solve a problem

- **Function**
  - A block of reusable code, a way of grouping related steps

- **Data Structure**
  - A particular way of organizing data in a computer

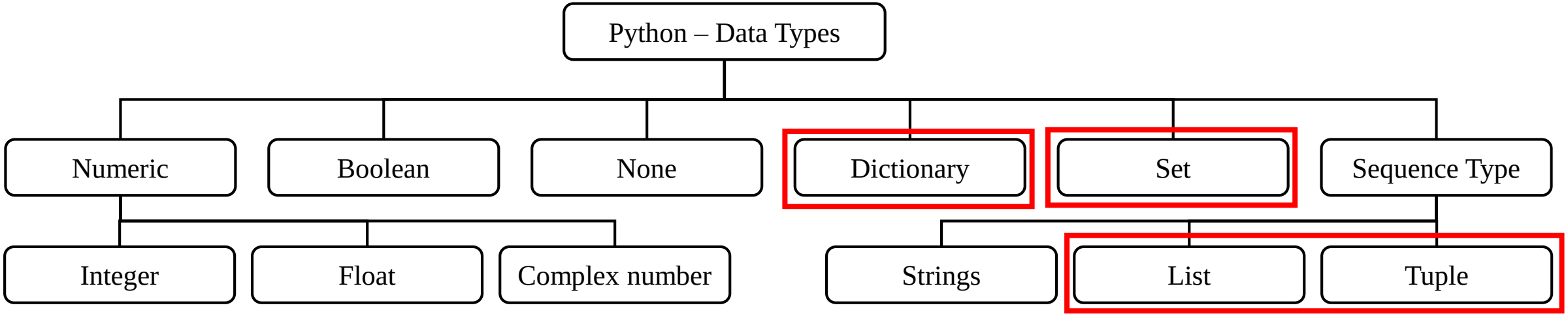



```
# define functions
def read_list():
    …

def sort_list():
    …

def print_list():
    …

# main body
read_list()
sort_list()
print_list()
```

https://en.wikipedia.org/wiki/Algorithm
https://en.wikipedia.org/wiki/Data_structure

Standard data types in Python

- A data type defines the kind or category of data (integer, float, string, etc)
- We can assign various types of data to a variable.



Data types	Examples	Data types	Examples	Data types	Examples
Integer	…, -2, -1, 0, 1, ...	Bool	True, False	Set	{1, 0.3, “a”, “ab”}
Float	3.2, -0.14, 35.75	String	“Hi, data!”, “123”	List	[1, 0.3, “a”, “ab”]
Complex	1+3j	Dictionary	{1:”a”, 2: “b”}	Tuple	(1, 0.3, “a”, “ab”)

What is Not a “Collection”?

Most of our variables have **one value** in them

➔ when we put a new value in the variable, the old value is gone

(e.g., Non-collection type: int, float, complex, bool, None, …)

```
$ python
>>> x = 2
>>> x = 4
>>> print(x)
```

Output:

```
4
```

What is a “Collection”?

Collection?

- A container (data structure) that **stores multiple values** in a single variable to organize, manage, and process data efficiently.

Collection data structure in Python

1. List
   - Ordered, mutable (can be changed)
   - Allows duplicate values
   - Useful when need a dynamic, resizable array

2. Tuple
   - Ordered, immutable (Cannot be changed)
   - Allows duplicate values.
   - Often used for fixed data or to ensure data integrity

```
# 1. list
fruits = ["apple", "banana", "cherry"]
numbers = [1, 3, 7, 13]
# mixed data type
mixed = [1, “abc”, 7, [1, 3, 4], x]
matrix = [[1,2], [3,4], [5,6]]

# 2. Tuple
point = (10.5, 20.3)
student_info = (“Kim”, 21, “EnergyAI”)
rgb = ((255,0,0), (0,255,0), (0,0,255))
# duplicated allow
repeated = (1,1,2,2,3)
```

What is a “Collection”?

Collection?

- A container (data structure) that **stores multiple values** in a single variable to organize, manage, and process data efficiently.

3. Set
   - Unordered, mutable (can add/remove item)
   - Does not allow duplicates.
   - Useful for membership test, removing duplicates

4. Dictionary
   - Unordered, mutable
   - Stores data as key-value pairs.
   - Keys must be unique
   - Useful for fast look-up

```
# 3. set
unique_number = {1, 2, 3, 3, 4}
print(unique_number)
> {1, 2, 3, 4}
letter = {“a”, “the”, “the”, “mail”}

print({1,3,5} | {3,5,7})
> {1,3,5,7} # union

# 4. Dictionary
student = {“name”:”Kim”, “age”: 21}
mixed = {1: “one”, “two”:2,
         3,4):”tuple as key”}

sample = {“a”:1, “a”:2}
print(sample)
> {“a”: 2}# last value overrides
```

A List is a kind of Collection

- A collection allows us to put many values in a single **“variable”**
- A collection is nice because we can carry all many values around in one convenient package.

```
friends = [ 'Joseph', 'Glenn', 'Sally' ]

carryon = [ 'socks', 'shirt', 'perfume' ]
```

List

- **List** is surrounded by **square brackets "[]"** and the elements in the list are separated by **commas ","**
  e.g., x = [1, "ai", 3], x= [], x=list((1,3))

- A **list** element can be **any Python object** : e.g., integer, string, variable, tuple, dictionary, object, function, even another list, …

- A **list** can be empty

```
>>> print([1, 24, 76])
[1, 24, 76]
>>> print(['red', 'yellow', 'blue'])
['red', 'yellow', 'blue']
>>> print(list(('red', 24, 98.6)))
['red', 24, 98.6]
>>> print([ 1, [5, 6], 7])
[1, [5, 6], 7]
>>> print([])
[]
```

We Already Use Lists!

```
for i in [5, 4, 3, 2, 1] :
    print(i)
print('Blastoff!')
```

Output:

5
4
3
2
1
Blastoff!

```
friends = ['Joseph', 'Glenn', 'Sally']
for friend in friends :
   print('Happy New Year:', friend)
print('Done!')
```

Output:

Happy New Year: Joseph
Happy New Year: Glenn
Happy New Year: Sally

Done!

Looking inside Lists

- Accessing elements
  : Just like strings, we can get at any single element in a list using **an index** specified in **square brackets []**

0	1	2
Joseph	Glenn	Sally
-3	-2	-1

```
>>> friends = [ 'Joseph', 'Glenn', 'Sally' ]
>>> print(friends[1])
Glenn
>>> print(friends[-1])
Sally
```

Lists are mutable

- Modifying List
  : Strings are “immutable” - we cannot change the contents of a string - we must make a new string to make any change

  : Lists are “mutable” - we can change an element of a list using the index operator

```
>>> fruit = 'Banana'
>>> fruit[0] = 'b'
Traceback
TypeError: 'str' object does not support item assignment
>>> x = fruit.lower()
>>> print(x)
banana
>>> lotto = [2, 14, 26, 41, 63]
>>> print(lotto)
[2, 14, 26, 41, 63]
>>> lotto[2] = 28
>>> print(lotto)
[2, 14, 28, 41, 63]
```

How long is a list?

- The len() function takes a list as a parameter and returns the number of **elements** in the list

- Actually len() tells us the number of elements of any set or sequence (such as a string...)

```
>>> greet = 'Hello Bob'
>>> print(len(greet))
9
>>> x = [ 1, 2, 'joe', 99]
>>> print(len(x))
4
>>>
```

List creation with the range function

- In Python2, the range function returns a list of numbers that range from zero to one less than the parameter
- We can create list with range function
- We can construct an index loop using **for** and an integer iterator

```
>>> print(range(4))
[0, 1, 2, 3] # Python 2
range(0, 4) # Python 3, range object

# quick list creation with range func.
>>> list_1 = list(range(4))
>>> squares = [x**2 for x in range(5)]
>>> print(squares)
[0, 1, 4, 9, 16)]

# construct index loop for loop statement
>>> friends = ['Joseph', 'Glenn', 'Sally']
>>> print(len(friends))
3
>>> print(list(range(len(friends))))
[0, 1, 2]
```

List operators & methods

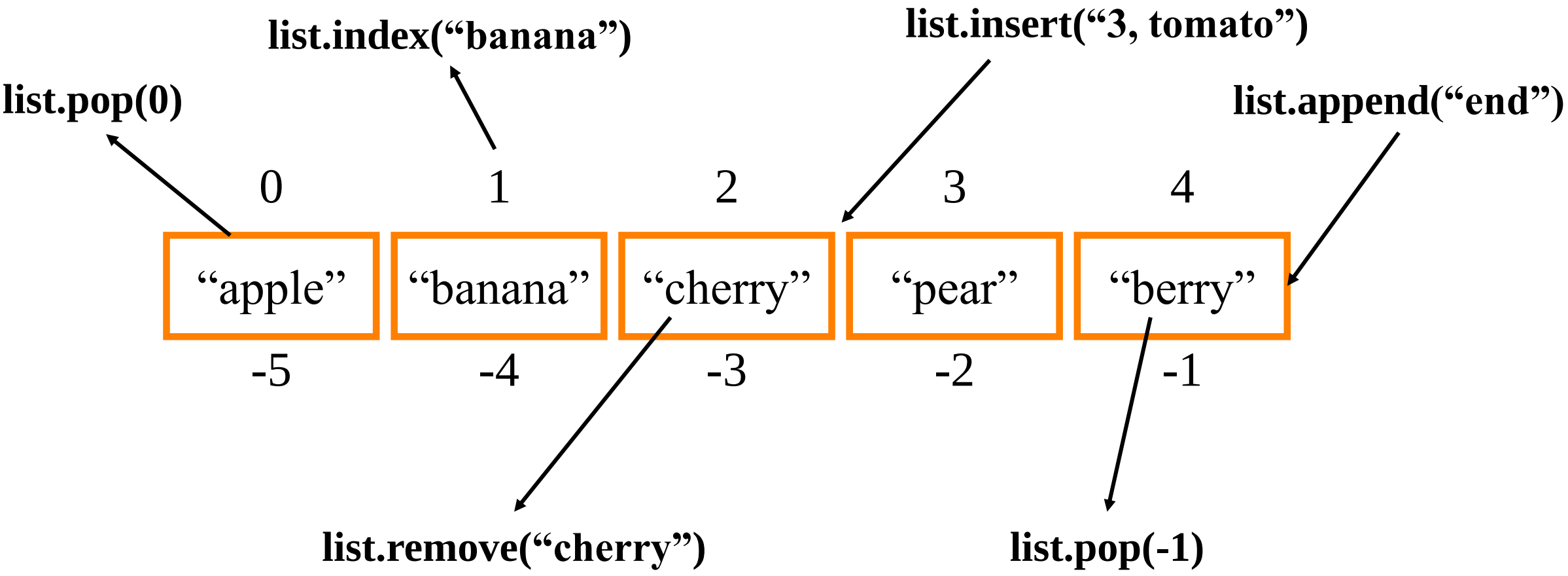
- Adding element (append, insert)
  : append(item) → the list stays in order and new elements are added at the end
  : insert(index, item) → add at a specific position

```
>>> stuff = list()
>>> stuff.append('book')
>>> stuff.append(99)
>>> print(stuff)
['book', 99]
>>> stuff.insert(0,'cookie')
>>> print(stuff)
['cookie','book', 99]
```

- Removing element (remove, pop,clear)
  : remove(item) ➔ remove by value
  : pop(index) ➔ remove by position
  : clear() ➔ empty the list

```
>>> some = [1, 3, 21, 10, 16]
>>> some.remove(1)
>>> print(some)
[3, 21, 10, 6]
>>> x = some.pop(3)
>>> print(some)
[3, 21, 10]
>>> print(x)
6
```

List operators & methods



List operators & methods

- Concatenating Lists using +
  : We can create a new list by adding two existing lists together

```
>>> a = [1, 2, 3]
>>> b = [4, 5, 6]
>>> c = a + b
>>> print(c)
[1, 2, 3, 4, 5, 6]
>>> print(a)
[1, 2, 3]
```

- Lists Can Be Sliced Using :
  : Just like in strings, the second number is "up to but not including"

```
>>> t = [9, 41, 12, 3, 74, 15]
>>> t[1:3]
[41,12]
>>> t[1:5:2] # start:stop:step, Q.::-2?
[41,3]
>>> t[:4]
[9, 41, 12, 3]
>>> t[3:]
[3, 74, 15]
>>> t[:]
[9, 41, 12, 3, 74, 15]
```

List operators & methods

- Repeating lists using *
  : We can create a new list by repeating lists

```
>>> a = [1, 2, 3]
>>> b = a * 3
>>> print(b)
[1, 2, 1, 2, 1, 2]
```

- Is something in a list? (in / not in)
  : Python provides two operators that let you check if an item is in a list
  : These are logical operators that return True or False
  : They do not modify the list

```
>>> some = [1, 9, 21, 10, 16]
>>> 9 in some
True
>>> 15 in some
False
>>> 20 not in some
True
```

List operators & methods

- Lists are in order (sort)
  : A list can hold many items and keeps those items in the order until we do something to change the order
  : A list can be sorted (i.e., change its order)
  : The sort method (unlike in strings) means "sort yourself"

- Other list operators & methods?
  : check with dir

http://docs.python.org/tutorial/datastructures.html

```
>>> friends = [ 'Joseph', 'Glenn',
'Sally' ]
>>> friends.sort()
>>> print(friends)
['Glenn', 'Joseph', 'Sally']
>>> print(friends[1])
Joseph
>>>
```

```
>>> x = list()
>>> type(x)
<type 'list'>
>>> dir(x)
[... 'append', 'count', 'extend',
'index', 'insert', 'pop', 'remove',
'reverse', 'sort']
```

Built-in Functions and Lists

- There are a number of functions built into Python that take lists as parameters

- Remember the loops we built? These are much simpler.

```
>>> nums = [3, 41, 12, 9, 74, 15]
>>> print(len(nums))
6
>>> print(max(nums))
74
>>> print(min(nums))
3
>>> print(sum(nums))
154
>>> print(sum(nums)/len(nums))
25.6
```

```python
total = 0
count = 0
while True :
    inp = input('Enter a number: ')
    if inp == 'done' : break
    value = float(inp)
    total = total + value
    count = count + 1
average = total / count
print('Average:', average)
```

```
Enter a number: 3
Enter a number: 9
Enter a number: 5
Enter a number: done
Average: 5.66666666667
```

```python
numlist = list()
while True :
    inp = input('Enter a number: ')
    if inp == 'done' : break
    value = float(inp)
    numlist.append(value)
average = sum(numlist) / len(numlist)
print('Average:', average)
```

Best Friends: Strings and Lists

```
>>> abc = 'With three words'
>>> stuff = abc.split()
>>> print(stuff)
['With', 'three', 'words']
>>> print(len(stuff))
3
>>> print(stuff[0])
With
```

```
>>> print(stuff)
['With', 'three', 'words']
>>> for w in stuff :
...     print(w)
...
With
Three
Words
>>>
```

Split breaks a string into parts and produces a list of strings. We think of these as words. We can **access** a particular word or **loop** through all the words.

Best Friends: Strings and Lists

```
>>> line = 'A lot                    of spaces'
>>> etc = line.split()
>>> print(etc)
['A', 'lot', 'of', 'spaces']
>>>
>>> line = 'first;second;third'
>>> thing = line.split()
>>> print(thing)
['first;second;third']
>>> print(len(thing))
1
>>> thing = line.split(';')
>>> print(thing)
['first', 'second', 'third']
>>> print(len(thing))
3
>>>
```

- When you do not specify a delimiter, multiple spaces are treated like one delimiter
- You can specify what delimiter character to use in the splitting

Best Double Split Pattern: Strings and Lists

- Sometimes we split a line one way, and then grab one of the pieces of the line and split that piece again

```
From stephen.marquard@uct.ac.za Sat Jan  5 09:14:16 2008

words = line.split()
email = words[1]
pieces = email.split('@')
print(pieces[1])
```

```
stephen.marquard@uct.ac.za
['stephen.marquard', 'uct.ac.za']
'uct.ac.za'
```

Lab 1. List

1. [Sum of input integers]

   Input: a single line of integers: e.g., 3, 4, 5, 6, 7, 8, 9

   Example input/output:

```
work@main1[esGNpu87-session]:~/EF2039_AI$ python W08_1_SumInteger.py
Enter a single line of integers: 3,4,5,6,7,8,9
Sum is 42
work@main1[esGNpu87-session]:~/EF2039_AI$ python W08_1_SumInteger.py
Enter a single line of integers: 6,4,3,7,6,4,1,2,4,7,5,2
Sum is 51
```

[Hint]
- Use input().split(“,”)
- Split() return a **list**
- Use int() for type converting
- Do not use sum() function

2. [Energy Meter CSV (sum per house)]

   Input: one-line CSV: <*house_id*, *kWh*> pairs separated by semicolon (“;”)
   ➔ “A,2.5 ; B,2.0 ; C,4.0 ; C, 5.0; A, 2.3; B, 1.5”

   Output: print total kWh per house

Example input/output:

```
work@main1[esGNpu87-session]:~/EF2039_AI$ python W08_2_EnergyMeterCSV.py
Enter one-line CSV file: A,2.5;B,2.0;C,4.0;C,5.0;A,2.3;B,1.5
A:4.8, B:3.5, C:9.0
```

[Hint]
- Use input() to get one-line CSV
- Use strip() to remove space
  : str = input(“enter one-line…”).strip()
- str.split(“,”) yields [house, value]
- Use split twice; semicolon, comma

Lab 1. List

3. [Random movie recommendation]

   Input: a single line of movies: e.g., “K-pop Demon Hunters”, “Modern Times”, “King”, “zzz”

   Output: random movie name

   Example input/output:

```
work@main1[fM6E18DE-session]:~/EF2039_AI$ python W08_3_RandomMovie.py
Enter movie list: “K-pop Demon Hunters”, “Modern Times”, “King”, “zzz”
Movie recommendation:  “Modern Times”
work@main1[fM6E18DE-session]:~/EF2039_AI$ python W08_3_RandomMovie.py
Enter movie list: “K-pop Demon Hunters”, “Modern Times”, “King”, “zzz”
Movie recommendation: “K-pop Demon Hunters”
```

[Hint]
- Use input().split(“,”)
- Split() return a **list**
- Use random.randint() function

4. [Find the Second Largest Number] Write a program with
   - Input: a single line of integers: e.g., -7, 2, 3, 8, 6, 6, 75, 38, 41
   - Function: find_second_max(num_list) ➔ return the second largest number

Example input/output:

```
work@main1[fM6E18DE-session]:~/EF2039_AI$ python W08_4_FindSecondLargest.py
Enter a single line of integers: -7, 2, 3, 8, 6, 6, 75, 38, 41
Second max = 41
work@main1[fM6E18DE-session]:~/EF2039_AI$
```

[Hint]
- Use list(map(int, list_str)
  or list_int = [int(i) for i in list_str]

Lab 1. List

5. [Matrix Transpose] Write a program that performs the transpose operation of a matrix.
   - Define a 2D list (matrix)
   - Used nested for loops to compute the transpose of the matrix

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}^T = \begin{bmatrix} 1 & 4 & 7 \\ 2 & 5 & 8 \\ 3 & 6 & 9 \end{bmatrix}$$

lst[0][0]	lst[0][1]	lst[0][2]
lst[1][0]	lst[1][1]	lst[1][2]
lst[2][0]	lst[2][1]	lst[2][2]

Example input/output:

```
work@main1[fM6E18DE-session]:~/EF2039_AI$ python W08_5_MatrixTranspose
Original matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
Transposed matrix = [[1, 4, 7], [2, 5, 8], [3, 6, 9]]
work@main1[fM6E18DE-session]:~/EF2039_AI$
```

[Hint]
- Use len(matrix), len(matrix[0]) to get # of rows, columns

Lab 1. List

6. [Data Cleaning: Filter Invalid Temperature Values] Write a program that cleans a list of temperature measurement. Some values are invalids, such as :
   - 100 (too high)
   - None (missing data)
   - “abc” (string value)

   Only valid temperatures within **30°C to 45°C** should remain in the cleaned list.

   Input: a single line of temp. measurements: 36.5, 37.0, 100, None, "abc", 34.2, 45.1, 39.8

   Output: cleaned and valid temp. measurements

Example input/output:

```
work@main1[fM6E18DE-session]:~/EF2039_AI$ python W08_6_DataClearning
Enter a single line of temp. measurement: 36.5, 37.0, 100, None, "abc", 34.2, 45.1, 39.8
Before cleaning: ['36.5', ' 37.0', ' 100', ' None', ' "abc"', ' 34.2', ' 45.1', ' 39.8']
After cleaning: [36.5, 37.0, 34.2, 39.8]
work@main1[fM6E18DE-session]:~/EF2039_AI$
```

[Hint]
- Use ***try-except*** to check type conversion ***float(data)*** is valid
- Use ***continue*** when it is exception cases

Summary

[List]

- List: ordered, mutable data structure
- Allows duplicate values
- Useful when need a dynamic, resizable array
- Creating list: x = [a,b,c]
- Lists and definite loops : for i in [a,b,c]
- Accessing : x[0], x[-1], x[0:3], x[0:-1:2]
- Modifying: x[3] = “abc” (mutable)
- Adding: x.append(3), x.insert(0, “Hi!”)
- Removing: x.remove(3), x.pop(0), x.clear()
- Operation: +, *, in, not in, …

```
# 1. list
# string
fruits = ["apple", "banana", "cherry"]
# numbers
numbers = [1, 3, 7, 13]
# mixed data type
mixed = [1, “abc”, 7, [1, 3, 4], x]
# nested list
matrix = [[1,2], [3,4], [5,6]]
# Any Python object
idk = [fruits, numbers, mixed, int]
```

Dictionary

What is a Collection?

Collection?

- A container (data structure) that **stores multiple values** in a single variable to organize, manage, and process data efficiently.

Collection data structure in Python

List

- Ordered, mutable
- Allows duplicate values
- for dynamic, resizable array

Tuple

- Ordered, immutable
- Allows duplicate values.
- for fixed data or data integrity

Set

- Unordered, mutable
- Does not allow duplicates
- for removing duplicates

Dictionary

- Unordered, mutable
- Stores key-value pairs
- Keys must be unique
- for fast look-up

A story of two collections

- **List**
  : A linear collection of **values**
  Lookup by position 0 .. length-1

- **Dictionary**
  : A linear collection of **key-value** pairs
  Lookup by "tag" or "key"

https://en.wikipedia.org/wiki/Index_card#/media/File:LA2-katalogkort.jpg
https://commons.wikimedia.org/wiki/File:Shelves-of-file-folders.jpg

Dictionaries

- **Dictionaries** are Python's most powerful data collection
- Dictionaries allow us to do **fast database-like operations** in Python
- Similar concepts in different programming languages
  - Associative Arrays - Perl / PHP
  - Map, TreeMap, or HashMap - Java
  - Dictionary, Sorted Dictionary - C# / .Net

Dictionary in Python

- Dictionary over time in Python
  - Prior to Python 3.7 dictionaries did not keep entries in the order of insertion
  - Python 3.7 (2018) and later dictionaries keep entries in the order they were inserted
  - "insertion order" is not "always sorted order"

- Below the abstraction
  - Python lists, dictionaries, and tuples are "abstract objects" designed to be easy to use
  - For now we will just understand them and use them and thank the creators of Python for making them easy for us
  - **Using Python collections is easy**. But, creating the code to support them is tricky and uses Computer Science concepts like dynamic memory, arrays, linked lists, hash maps and trees.
  - But that implementation detail is for a later course… maybe algorithm or data structure course?

Comparing List and Dictionary

- We append values to the end of a List and look them up by position

```
>>> cards = list()
>>> cards.append(12)
>>> cards.append(3)
>>> cards.append(75)
>>> print(cards)
[12, 3, 75]
>>> print(cards[1])
3
>>> cards[1] = cards[1] + 2
>>> print(cards)
[12, 5, 75]
```

- We insert values into a Dictionary using a key and retrieve them using a key

```
>>> cabinet = dict()
>>> cabinet['summer'] = 12
>>> cabinet['fall'] = 3
>>> cabinet['spring'] = 75
>>> print(cabinet)
{'summer': 12, fall': 3, spring': 75}
>>> print(cabinet['fall'])
3
>>> cabinet['fall'] = cabinet['fall'] +
2
>>> print(cabinet)
{'summer': 12, 'fall': 5, 'spring': 75}
```

Comparing List and Dictionary

Dictionaries are like **lists** except that
they use **keys** instead of **positions** to look up **values**

```
>>> lst = list()
>>> lst.append(21)
>>> lst.append(183)
>>> print(lst)
[21, 183]
>>> lst[0] = 23
>>> print(lst)
[23, 183]
```

```
>>> ddd = dict()
>>> ddd['age'] = 21
>>> ddd['course'] = 182
>>> print(ddd)
{'age': 21, 'course': 182}
>>> ddd['age'] = 23
>>> print(ddd)
{'age': 23, 'course': 182}
```

Creating & Accessing Dictionary

Creating dictionary

- Dictionary literals use curly braces {} and have **key** : **value** pairs
- You can make an **empty dictionary {}** using empty curly braces

```
>>> jjj = {'chuck' : 1 , 'fred' : 42, 'jan': 100}
>>> print(jjj)
{'chuck': 1, 'fred': 42, 'jan': 100}
>>> ooo = { }
>>> print(ooo)
{}
>>> dic = { x : x*x for x in range(6) if x%2 == 0}
{2: 4, 4: 16, 6: 36}
```

```
dic = {}
for x in range(6):
  if x%2 == 0:
      dic[x] = x*x
```

Accessing dictionary ([“key”] vs. get(key))

- **[“key”]** raise keyError if key missing
- **get(key, default)** returns default instead (safe lookups)

```
>>> jjj['kim']
KeyError: 'kim'
>>> print(jjj.get('Kim'))
None
>>> print(jjj.get('Kim',"N/A"))
N/A
```

Inserting & Updating & Deleting

Inserting & updating

- dict[“key”] = value
- dict.update({“a”:3, “b”:30})

Deleting

- dict.pop(“key”)
- key, value = Dict.popitem()
- del dict[“key”]

```
>>> x = {'a' : 1}
>>> x['b'] = 3                    # insert
>>> x['a'] = 10                   # update
>>> x.update({'c':4,'Kim':int}) # bulk update

>>> val = x.pop('b')              # remove key, returns value
>>> print(val)
3
>>> k, v = x.popitem()            # remove last inserted pair (LIFO)
Kim, <class 'int'>
>>> del x['c']                    # delete key
>>> x.clear()                     # empty dict
{}
```

Iteration patterns with key, value, boths

```python
scores = {"Alice": 95, "Bob": 88}

# keys (default)
for name in scores:
    print(name, scores[name])

# 1. explicit view for keys
for name in scores.keys():
    print(name)

# 2. explicit view for values
for score in scores.values():
    print(score)

# 3. explicit view for keys and values
for name, score in scores.items():
    print(name, score)
```

```
Output


Alice 95
Bob 88


Alice
Bob


95
88


Alice 95
Bob 88
```

Dictionary

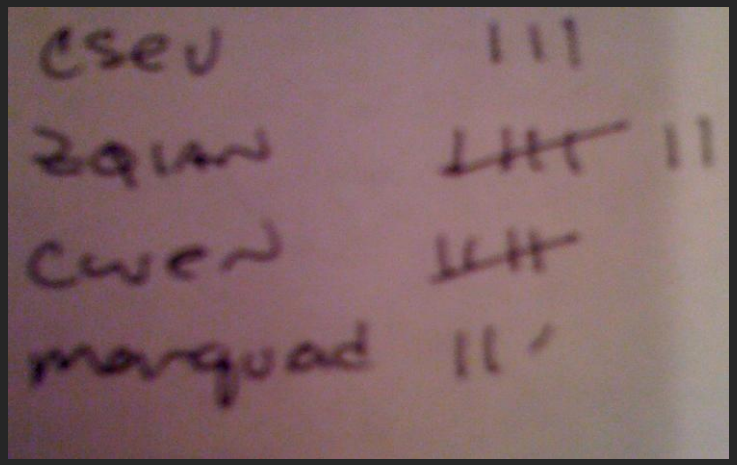
Example: Many counters with a dictionary

Example. Many counters with a Dictionary

One common use of dictionaries is counting how often we **“see”** something from **“data”**

```
>>> ccc = dict()
>>> ccc['csev'] = 1
>>> ccc['cwen'] = 1
>>> print(ccc)
{'csev': 1, 'cwen': 1}
>>> ccc['cwen'] = ccc['cwen'] + 1
>>> print(ccc)
{'csev': 1, 'cwen': 2}
```

Key Value



Dictionary Tracebacks

- It is an error to reference a key which is not in the dictionary

>> We can use the in operator or get function to see if a key is in the dictionary

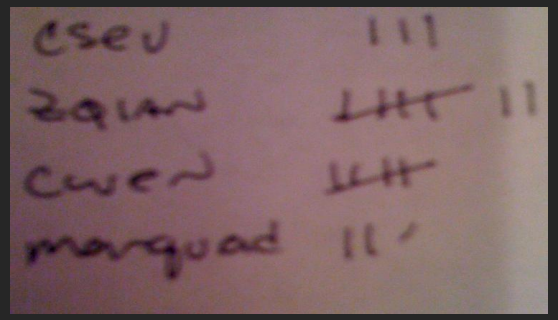
```
>>> ccc = dict()
>>> print(ccc['csev'])
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
KeyError: 'csev'
>>> 'csev' in ccc
False
```

When we see a new name

When we encounter a new name, we need to add a new entry in the dictionary and if this the second or later time we have seen the name, we simply add one to the count in the dictionary under that name

```
counts = dict()
names = ['csev', 'cwen', 'csev', 'zqian', 'cwen']
for name in names :
    if name not in counts:
        counts[name] = 1
    else :
        counts[name] = counts[name] + 1
print(counts)
```

{'csev': 2, 'cwen': 2 , 'zqian': 1}



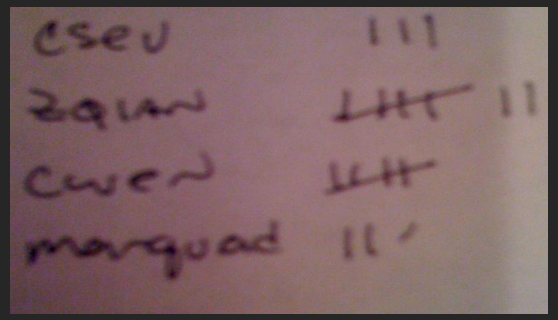
Simplified Counting with get()

We can use get() and provide a default value of zero when the key is not yet in the dictionary - and then just add one

```
counts = dict()
names = ['csev', 'cwen', 'csev', 'zqian', 'cwen']
for name in names :
    counts[name] = counts.get(name, 0) + 1
print(counts)
```

Default

{'csev': 2, 'cwen': 2 , 'zqian': 1}



Counting words in text

Writing programs (or programming) is a very creative and rewarding activity. You can write programs for many reasons ranging from making your living to solving a difficult data analysis problem to having fun to helping so meone else solve a problem. This book assumes that everyone needs to know how to program and that once you know how to program, you will figure out what you want to do with your newfound skills.

We are surrounded in our daily lives with computers ranging from laptops to cell phones. We can think of these computers as our “personal assistants” who can take care of many things on our behalf. The hardware in our cur rent-day computers is essentially built to continuously ask us the question, “What would you like me to do next? ”

Our computers are fast and have vast amounts of memory and could be very helpful to us if we only knew the la nguage to speak to explain to the computer what we would like it to do next. If we knew this language we could tell the computer to do tasks on our behalf that were repetitive. Interestingly, the kinds of things computers can d o best are often the kinds of things that we humans find boring and mind-numbing.

Counting words in text

The general pattern to count the words in a line of text is to split the line into words, then loop through the words and use a dictionary to track the count of each word independently.

```
counts = dict()
print('Enter a line of text:')
line = input('')

words = line.split()

print('Words:', words)

print('Counting...')
for word in words:
    counts[word] = counts.get(word,0) +
1
print('Counts', counts)
```

Counting words in text

```
python wordcount.py
Enter a line of text:
the clown ran after the car and the car ran into the tent and the tent fell down
on the clown and the car

Words: ['the', 'clown', 'ran', 'after', 'the', 'car', 'and', 'the', 'car', 'ran',
'into', 'the', 'tent', 'and', 'the', 'tent', 'fell', 'down', 'on', 'the', 'clown',
'and', 'the', 'car']
Counting…

Counts {'the': 7, 'clown': 2, 'ran': 2, 'after': 1, 'car': 3, 'and': 3, 'into': 1,
'tent': 2, 'fell': 1, 'down': 1, 'on': 1}
```

Lab 2. Dictionary

1. [Parse colon pairs to dictionary]

   Input: “name:alice age:21 dept:AI”

   Example input/output:

```
work@main1[esGNpu87-session]:~/EF2039_AI$ python W08_3_ParseColon.py
Enter input text: name:alice age:21 dept:AI
{'name': 'alice', 'age': '21', 'dept': 'AI'}
work@main1[esGNpu87-session]:~/EF2039_AI$ python W08_3_ParseColon.py
Enter input text: name:Kim age:23 dept:AI
{'name': 'Kim', 'age': '23', 'dept': 'AI'}
```

[Hint]
- Split() by space
- Then, you get a list of [“name:alice”, “age:21”, …]
- Use split(“:”), it will give a list of [“name”, “Alice”]

2. [Group numbers by parity]

   Input: 3 4 5 6 7 8

   Output: {‘even’ : [4, 6, 8], ‘odd’:[3, 5, 7]}

   Example input/output:

```
work@main1[esGNpu87-session]:~/EF2039_AI$ python W08_4_GroupNumber.py
Enter your number set: 3 4 5 6 7 8
{'even': [4, 6, 8], 'odd': [3, 5, 7]}
work@main1[esGNpu87-session]:~/EF2039_AI$ python W08_4_GroupNumber.py
Enter your number set: 1 78 83 92 100 322 3233
{'even': [78, 92, 100, 322], 'odd': [1, 83, 3233]}
```

[Hint]
- Read ints, group into a dict with keys ‘even’ and ‘odd’

```
num = ...
groups = {'even': [], 'odd': []}
for n in nums:
    ...
print(groups)
```

Lab 2. Dictionary

3. [Address Book] Build a command-line address book that lets the user manage contacts
   - Add contact
     : ask name, then phone number
     : save to a dictionary (key = name, value = phone)
     : if the name exists, update the phone number
   - Delete contact
     : ask name to delete
     : if found, remove the item, otherwise print “Not found”
   - Search contact
     : ask name to search
     : if found, print: <NAME>’s number: <PHONE>
     else print “Not found”
   - Print contacts
     : print all saved contacts as Name:Phone, sorted by name.
     : if empty, print No contacts
   - Exits
     : end the program

Example input/output:

```
work@main1[1yaCpTcp-session]:~/EF2039_AI$ python W08_9_AddressBook.py
1. Add contact
2. Delete contact
3. Search contact
4. Print contacts
5. Exit
Select a menu item: 1
Name: Kim
Phone number: 010-1234-5678
Added Kim.
1. Add contact
2. Delete contact
3. Search contact
4. Print contacts
5. Exit
Select a menu item: 4
Kim: 010-1234-5678
```

[Hint]
- Use sorted() to sort by name
- Use functions to make code readable

Lab 2. Dictionary

3. [Address Book] Build a command-line address book that lets the user manage contacts

Example input/output:

```
work@main1[1yaCpTcp-session]:~/EF2039_AI$ python W08_9_AddressBook.py
1. Add contact
2. Delete contact
3. Search contact
4. Print contacts
5. Exit
Select a menu item: 1
Name: Kim
Phone number: 010-1234-5678
Added Kim.
1. Add contact
2. Delete contact
3. Search contact
4. Print contacts
5. Exit
Select a menu item: 4
Kim: 010-1234-5678
```

```
1. Add contact
2. Delete contact
3. Search contact
4. Print contacts
5. Exit
Select a menu item: 4
Kim: 010-1234-5678
Lee: 061-320-1234
Park: 02-2201-1111
```

```
Select a menu item: 3
Name to search: Kim
Kim's number: 010-1234-5678
```

```
Select a menu item: 3
Name to search: Joo
Not found
```

```
Select a menu item: 1
Name: Kim
Phone number: 010-5678-1234
Updated Kim's number.
```

```
Select a menu item: 2
Name to delete: Lee
Deleted Lee.
1. Add contact
2. Delete contact
3. Search contact
4. Print contacts
5. Exit
Select a menu item: 4
Kim: 010-5678-1234
Park: 02-2201-1111
```

```
1. Add contact
2. Delete contact
3. Search contact
4. Print contacts
5. Exit
Select a menu item: 5
Bye!
```

Summary

[Dictionary]

- Dictionary
  : Unordered, mutable data structure
  : Stores data as key-value pairs
  : Key must be unique
  : Useful for fast look-up
- Creating dict
  : x = {‘key1’: value, ‘key2, “value”, …}
- Accessing : x[“key”], x.get(“key”)
- Adding&Updating
  : x[“key”] = “abc” (mutable)
  : x.update({“key”:value})
- Removing: x.pop(“key”), del x[“key”], x.clear()

```
# 2. Dictionary (key-value)
student = {“name”:”Kim”, “age”: 21}
mixed = {1: “one”, “two”:2,
         3,4):”tuple as key”}

sample = {“a”:1, “a”:2}
print(sample)
> {“a”: 2}# last value overrides

robot = {
    "id": "rx01",
    "sensors": {
        "cam": {"type": "rgb",
                "fps": 30},
        "therm": {"type": "thermal",
                  "range_m": 50}
    }
}
```

Tuple

List vs. Tuple

Collection data structure in Python

1. List
   - Ordered, mutable (can be changed)
   - Allows duplicate values
   - Useful when need a dynamic, resizable array

2. Tuple
   - Ordered, immutable (Cannot be changed)
   - Allows duplicate values.
   - Often used for fixed data or to ensure data integrity
   - Hashable iff <u>all elements are hashable</u>
     ➔ usable as dict keys /set elements

```
# 1. list
fruits = ["apple", "banana", "cherry"]
numbers = [1, 3, 7, 13]
# mixed data type
mixed = [1, “abc”, 7, [1, 3, 4], x]
matrix = [[1,2], [3,4], [5,6]]

# 2. Tuple
point = (10.5, 20.3)
student_info = (“Kim”, 21, “EnergyAI”)
rgb = ((255,0,0), (0,255,0), (0,0,255))
# duplicated allow
repeated = (1,1,2,2,3)
```

Tuples are like lists

Tuples are another kind of sequence that functions much like a list - they have elements which are indexed starting at 0

```
>>> x = ('Glenn', 'Sally', 'Joseph')
>>> print(x[2])
Joseph
>>> y = ( 1, 9, 2 )
>>> print(y)
(1, 9, 2)
>>> print(max(y))
9
```

```
>>> for iter in y:
...     print(iter)
...
1
9
2
>>>
```

But… Tuples are “immutable”

Unlike a list, once you create a tuple, you cannot alter its contents - similar to a string

```
# list
>>> x = [9, 8, 7]
>>> x[2] = 6
>>> print(x)
>>>[9, 8, 6]
>>>
```

```
# string
>>> y = 'ABC'
>>> y[2] = 'D'
Traceback:'str' object
does
not support item
Assignment
>>>
```

```
# tuple
>>> z = (5, 4, 3)
>>> z[2] = 0
Traceback:'tuple'
object does
not support item
Assignment
>>>
```

Things not to do with Tuples

Unlike a list, once you create a tuple, you cannot alter its contents - similar to a string

```
>>> x = (3, 2, 1)
>>> x.sort()
Traceback:
AttributeError: 'tuple' object has no attribute 'sort'
>>> x.append(5)
Traceback:
AttributeError: 'tuple' object has no attribute 'append'
>>> x.reverse()
Traceback:
AttributeError: 'tuple' object has no attribute 'reverse'
>>>
```

A Tale of Two Sequences (List vs. Tuple)

```
>>> l = list()
>>> dir(l)
['append', 'count', 'extend', 'index', 'insert', 'pop', 'remove', 'reverse',
'sort']

>>> t = tuple()
>>> dir(t)
['count', 'index']
```

Tuples and Assignment

- You can make an **empty tuple** using empty parentheses “( )”
- We can also put a **tuple** on the **left-hand side** of an assignment statement
- We can even omit the parentheses

```
>>> empty = ()                    # empty tuple
>>> one = (42, )                  # single-element needs trailing comma!
>>> also_one = 42,                # parentheses optional; the comma makes the tuple
>>> from_iter = tuple([1,2,3]) # constructor
>>> nested = ((1,2), (3,4))
>>> t = (4, 'fred', 3.14)
>>> x, y  = 1, 3
>>> x, y, z = t                   # unpacking
```

Accessing & Operation

- Accessing
  : tuple[index]
- Slicing
  : tuple[start:end:step]
- Concatenation
  : tuple + tuple
- Repetition
  : tuple*2
- Find index
  : tuple.index(value)

```
>>> t = ("r", "g", "b", "ir")
>>> print(t[0], t[-1])
r, ir
>>> print(t[1:3])
('g', 'b')

t = (1, 2, 2, 3)
print(len(t))                # 4
print(t + (4, 5))            # concatenation
print(t * 2)                 # repetition
print(2 in t)                # membership
print(t.count(2))            # 2
print(t.index(3))            # 3
```

Tuples are comparable

The comparison operators work with tuples and other sequences. If the first item is equal, Python goes on to the next element, and so on, until it finds elements that differ.

```
>>> (0, 1, 2) < (5, 1, 2)
True
>>> (0, 1, 2000000) < (0, 3, 4)
True
>>> ( 'Jones', 'Sally' ) < ('Jones', 'Sam')
True
>>> ( 'Jones', 'Sally') > ('Adams', 'Sam')
True
```

Tuples are more efficient

- Since Python does not have to build tuple structures to be modifiable, they are simpler and **more efficient in terms of memory use and performance** than lists
- So in our program <u>when we are making “temporary variables” we prefer tuples</u> over lists

Tuples and Dictionary

The items() method in dictionaries returns a list of (key, value) tuples

```
>>> d = dict()
>>> d['csev'] = 2
>>> d['cwen'] = 4
>>> for (k,v) in d.items():
...     print(k, v)
...
csev 2
cwen 4
>>> tups = d.items()
>>> print(tups)
dict_items([('csev', 2), ('cwen', 4)])
```

Common Use Cases (Immutability)

- **Fixed** records or units or contents: coordinates, RGB, dates, DB rows.
  : e.g., point = (x, y), size = (1024, 2048), pixel = (r, g, b), (year, month, day)…

- **Multiple returns** from functions.
  : ensure immutability

```
def bbox() -> tuple[int, int, int, int]:
    return (x_min, y_min, x_max, y_max)
```

- **(Hash) Keys** for dicts/sets (e.g., grid indices (row, col)).
  : e.g., dict[“tuple”] ➔ dict[(0,1)], dict[(1,1)], …

- **Avoid accidental mutation** vs lists.
  : e.g., return value, global variable, default parameter, etc…

Lab 3. Tuple

1. [Key by 2D index]

Task: Build a dictionary counter that counts visiting number to grid cells from a list of (row, col) tuple

Input: visits = [(0,0), (1,2), (0,0), (1,2), (1,2), (2,2)]

Example input/output:

```
work@main1[esGNpu87-session]:~/EF2039_AI$ python W08_5_Keyby2DIndex.py
{(0, 0): 2, (1, 2): 3, (2, 2): 1}
```

W08_5_Keyby2DIndex.py

```python
visits = [(0,0), (1,2), (0,0), (1,2), (1,2), (2,2)]

counts = {}
for cell in visits:
    ...
print(counts)
```

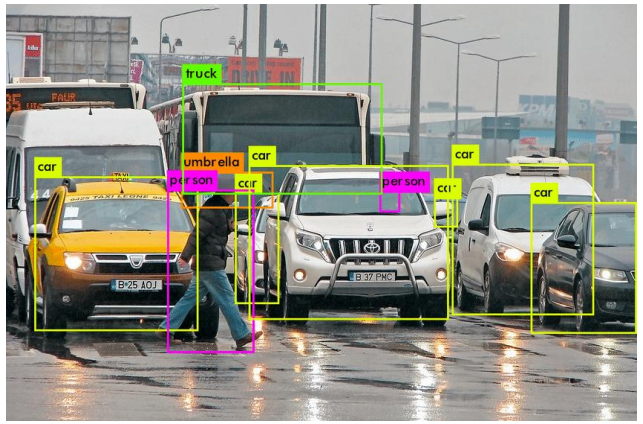
[Hint]

- Utilize tuple as key ➔ count[cell]

Lab 3. Tuple

2. [Tuples, Lists, Dicts] Object Detection - Bounding Boxes & Class Count

   Given a list of detections as tuples (***cls, x_min, y_min, x_max, y_max***),
   1. compute the area of each box,
   2. print a dict {cls: count},
   3. return the **largest-area** detection tuple.



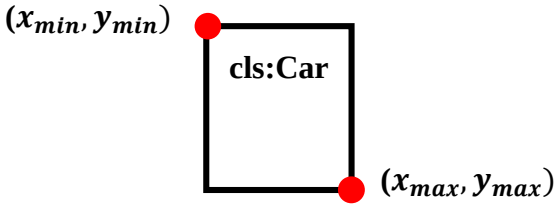
Example: a list of detection results

W10_1_ObjectDetection.py

```python
detections = [
    ("person", 10, 10, 50, 70),
    ("car", 0, 0, 100, 40),
    ("person", 120, 120, 160, 180),
    ("car", 30, 40, 80, 70),
    ("truck", 50, 80, 100, 150),
    ("person", 20, 20, 60, 80),
]
```

Example input/output:

```
work@main1[wGM4GbYQ-session]:~/EF2039_AI$ python W10_1_ObjectDetection.py
{'person': 3, 'car': 2, 'truck': 1}
Largest: ('car', 0, 0, 100, 40)
```



Hints

- Unpack tuples.
- Dict counting: counts[cls] = counts.get(cls, 0) + 1.
- Area: (x_max - x_min) * (y_max - y_min).

Follow-ups

Q. How to skip invalid boxes (negative width/height)?

Q. Why are tuples good for fixed records?

Summary

[Tuple]

- Tuple
  : Ordered, **immutable** data structure
  : Allows duplicate values
- Creating tuple
  : x = (1, 2, 3), x, y = 5, 7
- Accessing : x[0], x[-1], x[start:end:step]
- Use cases
  : fixed record or unit, hash key, multi-returns, avoiding mutation

```
# 3. Tuple
point = (10.5, 20.3)
student_info = ("Kim", 21, "EnergyAI")
rgb = ((255,0,0), (0,255,0), (0,0,255))
size = (512, 256)

# duplicated allow
repeated = (1,1,2,2,3)
nested = ((1,2), (3,4))

# packing & unpacking
t = 1, 5, "a"
x, y  = 1, 3
x, y, z = t

# key for dictionary
dict = {}
dict[(0,1)] = 0
```

Set

List vs. Set

Collection data structure in Python

1. List
   - Ordered, mutable (can be changed)
   - Allows duplicate values
   - Useful when need a dynamic, resizable array

2. Set
   - Unordered, mutable (can add/remove item)
     ➔ no indexing/slicing (order not guaranteed)
   - Does not allow duplicates.
   - Useful for membership test, removing duplicates

```
# 1. list
fruits = ["apple", "banana", "cherry"]
numbers = [1, 3, 7, 13]
# mixed data type
mixed = [1, “abc”, 7, [1, 3, 4], x]
matrix = [[1,2], [3,4], [5,6]]

# 3. set
unique_number = {1, 2, 3, 3, 4}
print(unique_number)
> {1, 2, 3, 4}
letter = {“a”, “the”, “the”, “mail”}

print({1,3,5} | {3,5,7})
> {1,3,5,7} # union
```

Creating Sets

- What is a set
  : Unordered, mutable collection of a unique hashable elements
  : No indexing/slicing

- Creating Sets
  : Elements must be hashable (e.g., int, str, tuple)
  : Lists/dicts/sets (mutable) are not hashable

- Set comprehensions

```
>>> empty = set()# correct empty set
# {} is an empty dictionary, not a set!
>>> techs = {'ai', 'robotics', 'energy'}
>>> idxs = {(0,0), (0,1), (1,1)}

>>> s1 = {1, 2, 3}
>>> s2 = set([1, 2, 2, 3])
>>> print(s2)
{1, 2, 3}

>>> s3 = set('robot')
>>> print(s3)
{'r', 'o', 'b', 't'}

>>> squares = {x*x for x in range(6)}
{0, 1, 4, 9, 16, 25}
>>> squares2 = {x*x for x in range(6)if
x%2==0}
{0, 16, 4}
```

Set operators & methods

- Adding
  : set.add(value)
  : set.update([value, value])
  > in-place union (|)

- Removing
  : set.remove(value)
  > KeyError if absent
  : set.discard(value)
  > safe: no error
  : set.pop()
  > remove & return arbitrary element
  : set.clear()

```
>>> s = {1, 2}
>>> s.add(3)
>>> print(s)
{1, 2, 3}
>>> s.update([3,4])
>>> print(s)
{1, 2, 3, 4}
```

```
>>> s.remove(5)    # KeyError if absent
>>> s.discard(5)   # safe: no error
>>> s.discard(4)   # safe: no error
>>> print(s)
{1, 2, 3}
>>> x = s.pop()    # remove & return
arbitrary element
>>> s.clear()
>>> print(s)
Set()
```

Set operators & methods

- Set operations (return sets)
  : A.union(B) or A | B
  : A.intersection(B) or A & B
  : A.difference(B) or A - B
  : A.symmetric_difference(B) or A ^ B
  = (A | B) – (A & B)

- Membership (return True/False)
  : A.issubset(B) or A > B # A ⊆ B
  : A.issuperset(B) or A < B # A ⊇ B
  : A.isdisjoint(B) # ∩ = ∅

- refer to more methods with **dir(set)**

```
>>> A = {1, 2, 3}
>>> B = {3, 4, 5}
>>> A | B          # union: {1,2,3,4,5}
>>> A.union(B)     # union: {1,2,3,4,5}
>>> A & B          # intersection: {3}
>>> A - B          # difference: {1,2}
>>> A ^ B          # symmetric difference
                   # :{1,2,4,5}
```

```
>>> C = {1, 2}
>>> A.issubset(C) # A ⊆ B, False
>>> A.issuperset(C) # A ⊇ B, True
>>> A.isdisjoint(C) # ∩ = ∅, False

>>> "lidar" in {"cam", "lidar", "imu"}
True
>>> for item in {"a", "b", "c"}:
... print(item)  # order not guaranteed
b, a, c
```

Common Use Cases

1. De-duplication

```
>>> names = ["cam","lidar","cam","imu","cam"]
>>> unique = set(names)                    # fast, but loses order
>>> print(unique)
["lidar", "cam", "imu"]

# order-preserving:
>>> seen, ordered = set(), []
>>> for n in names:
...     if n not in seen:
...         seen.add(n); ordered.append(n)
>>> print(seen)
["lidar", "cam", "imu"]
>>> print(ordered)
["cam", "lidar", "imu']
```

Common Use Cases

2. Fast membership filter

```
>>> filter = [“the",“a",“of”,“and"]
>>> token = “the robot of energy ai”.split()
>>> kept = [t for t in token if t not in filter]
>>> print(kept)
[“robot”, “ai”, “energy”]
```

Common Use Cases

3. Set logic in AI/Robotics

```
# Sensors available vs required
>>> available = {"cam","imu","gps"}
>>> required  = {"cam","lidar"}
>>> missing = required - available
>>> print(missing)
{“lidar”}

# Label filter
>>> predicted = {"car","bus","person"}
>>> allowed   = {"car","person"}
>>> invalid = predicted - allowed        # {'bus’}
>>> print(invalid)
{“bus”}
```

Lab 4. Set

1. [Valid Labels] Given allowed_label = {“cat, “dog”, “person”} and comma-separated prediction input, print the valid labels as a set.

   Input: cat, dog, car, truck, cat, person

Example input/output:

```
work@main1[1yaCpTcp-session]:~/EF2039_AI$ python W08_10_ValidLabel.py
Enter neural network's predictions: cat, dog, car, truck, cat, person
Valid prediction labels:  {'cat', 'person', 'dog'}
```

[Hint]
- Keep in mind the data type!
- Utilize str.replace(“ ”, “”) to remove all spacing
- Use intersection operation of sets.

2. [Missing Sensors] Find missing sensor from the space-separated input text, print the missing sensors.

   Requirement: {“RGB_camera”, “lidar”, “imu”, “depth_camera”}

   Input: RGB_camera lidar motor encoder

Example input/output:

```
work@main1[1yaCpTcp-session]:~/EF2039_AI$ python W08_11_MissingSensor.py
Enter current sensor list:  RGB_camera lidar motor encoder
Missing sensor list:  {'imu', 'depth_camera'}
```

Lab 4. Set

3. [Plagiarism Checker] Ask the user for two sentences and report how similar they are
    1. Read two input lines from the user
    2. Process each sentence: Convert to lowercase and split into word token
    3. Convert each token list to a set of unique words
    4. Compute similarity: ((A&B) / (A|B))*100
    5. Print the similarity percentage

[Hint]
- Use str.lower() to convert all characters to lowercase characters.

- First sentence: Alice was beginning to get very tired of sitting by her sister on the bank
- Second sentence: Kim was beginning to get very tired of sitting by her sister on the bank

Example input/output:

```
work@main1[1yaCpTcp-session]:~/EF2039_AI$ python W08_11_Plagiarism_Checker.py
First sentence: Alice was beginning to get very tired of sitting by her sister on the bank
Second sentence: Kim was beginning to get very tired of sitting by her sister on the bank

Similarity = 87.500000
Shared words (14): ['bank', 'beginning', 'by', 'get', 'her', 'of', 'on', 'sister', 'sitting', 'the', 'tired', 'to', 'very', 'was']
Only in #1   (1): ['alice']
Only in #2   (1): ['kim']
```

Lab 4. Set

4. [Stopword Filter] Natural Language Processing;
    1. Get stopwords as a first line input
    2. Then, get a sentence as a second input
    3. Print the unique words and word count list except stopwords
    4. Print the most frequent word except stopwords.
    - Input (stopwords): a, the, of, and
    - Input (sentence): ***stopwords filtering program: get stopwords as first line input, then, get a sentence as a second input, and then, print the most frequent word except stopwords.***

[Hint]
- Use replace() to remove “:”, “,”, “.”
- Use set to find unique words
- Use dictionary to count words

Example input/output:

```
work@main1[1yaCpTcp-session]:~/EF2039_AI$ python W08_12_WordCounter.py
Enter stopwords: a, the, of, and
Enter sentence: stopwords filtering program: get stopwords as first line input, then, get a sentence as a second input, and then, p
rint the most frequent word except stopwords.
============================
unique words except stopwords:  {'except', 'second', 'then', 'program', 'print', 'frequent', 'get', 'sentence', 'first', 'word', 'l
ine', 'as', 'stopwords', 'input', 'filtering', 'most'}
============================
word counts:  {'stopwords': 3, 'filtering': 1, 'program': 1, 'get': 2, 'as': 2, 'first': 1, 'line': 1, 'input': 2, 'then': 2, 'sent
ence': 1, 'second': 1, 'print': 1, 'most': 1, 'frequent': 1, 'word': 1, 'except': 1}
============================
word counts sorted by value:  [('filtering', 1), ('program', 1), ('first', 1), ('line', 1), ('sentence', 1), ('second', 1), ('print
', 1), ('most', 1), ('frequent', 1), ('word', 1), ('except', 1), ('get', 2), ('as', 2), ('input', 2), ('then', 2), ('stopwords', 3)
]
============================
The most frequent words:  ('stopwords', 3)
```

Summary

[Set]

- Set
  : Unordered, mutable data structure
  ➔ no indexing/slicing (order not guaranteed)
  : Does not allow duplicates
  : Useful for membership test, removing duplicates
- Creating set
  : x = set(), {value, value2, …}
  ➔ {} is dictionary
- Adding : x.add(value), x.update([value,value])
- Removing : x.remove(value), x.discard(value)
  x.pop(), x.clear()
- Set operation: |, &, -, ^, >, <

```
# 4. Set
unique_number = {1, 2, 3, 3, 4}
print(number_number)
> {1, 2, 3, 4}
letter = {"a", "the", "the", "mail"}
print(letter)
> {"the", "a", "mail"}

print({1,3,5} | {3,5,7})
> {1,3,5,7} # union
print({1,3,5} & {3,5,7})
> {3,5} # intersection
print({1,3,5} - {3,5,7})
> {1} # difference
print({1,3,5} ^ {3,5,7})
> {1,7} # symmetric difference
```

Q&A